

STM32 FreeRTOS Multisensor Monitor

Name: Rabago, Sean Marie G

Course: BCA182

Platform: STM32F103C8T6 Blue Pill, STM32Cube HAL, FreeRTOS, Wokwi

Repository: `bca182-freertos-multisensor`

Report status: Draft; Wokwi functional observations and fault-experiment results remain to be recorded.

1. Problem and Requirements

The project implements a small sensor-monitoring system on an STM32 Blue Pill. It must periodically read a DHT22 temperature/humidity sensor and an LDR through the ADC, display values on an SSD1306 OLED, accept page changes from a KY-040 rotary encoder, classify temperature alarms, and use a PIR sensor to control an ACTIVE/INACTIVE system state. The system is simulated in Wokwi, so peripheral timing, pin mappings, and the serial monitor are part of verification.

Key behavior includes a two-second sensor sampling period, a 15-second inactivity timeout, motion-triggered reactivation, four OLED pages, LOW/HIGH temperature alarm states outside the 18-30 C normal range, and readable diagnostics from concurrent tasks. LDR output is a normalized ADC percentage, not a calibrated lux value.

2. System Architecture and Design

Hardware and Wokwi Pin Configuration

Device/function	STM32 pin(s)	Implementation
DHT22 data	PB0	Timed single-wire pulse protocol; temperature and humidity returned after checksum validation.
LDR analog output	PA0	ADC1 channel 0; 12-bit raw count normalized to 0-100%.
PIR output	PA3	Digital input sampled by MotionTask.
Encoder CLK / DT	PA4 / PA5	Quadrature transitions decoded as detents.
OLED SDA / SCL	PB7 / PB6	I2C1 SSD1306 at address 0x3C.
USART1 TX / RX	PA9 / PA10	Serial monitor diagnostics.
Buzzer	PB10	TIM2 channel 3 PWM output.

The Wokwi circuit definition is `diagram.json`; Wokwi loads the ELF and binary referenced by `wokwi.toml` from `.pio/build/bluepill\_f103c8/`.

Required visual evidence: Insert a full Wokwi circuit screenshot here after capturing it in the simulator. Caption: *Figure 1. Wokwi circuit and pin wiring for the multisensor monitor.*

Software Decomposition

`main.cpp` initializes UART/hardware and RTOS objects, creates tasks, and starts the scheduler. Application behavior is separated into sensor, display, input, alarm, motion, and

system-state modules. Hardware access is isolated behind `hardware.h`; SSD1306 rendering is in the OLED module; deterministic alarm, navigation, and state decisions have hardware-independent functions.

```

```mermaid
flowchart LR
 DHT[DHT22] --> ST[SensorTask]
 LDR[LDR and ADC] --> ST
 PIR[PIR] --> MT[MotionTask]
 ENC[KY-040] --> IT[InputTask]
 ST --> DQ[Display sensor queue]
 ST --> AQ[Alarm sensor queue]
 DQ --> DT[DisplayTask]
 MQ[Display mode queue] --> DT
 SQ[System state queue] --> DT
 IT --> MQ
 MT --> SQ
 MT --> EG[Event group]
 AT[AlarmTask] --> EG
 EG --> IT
 EG --> DT
 AQ --> AT
 DT --> OLED[SSD1306]
 AT --> BUZZER[Buzzer]
```

```

Figure 2. Sensor inputs are sampled by dedicated tasks and communicated to their consumers using separate queues. Only DisplayTask accesses the OLED.

3. FreeRTOS Architecture

The kernel runs with preemption and a 100 Hz TIM3 tick. `SensorTask`, `InputTask`, and `MotionTask` use periodic delays; queue consumers block when no data is available. The custom Cortex-M3 port performs direct thread-mode context switches and does not switch task context inside the timer ISR.

| Task | Responsibility | Trigger / period | Priority | IPC | Typical blocked condition |
|-------------|--|------------------------------|----------|--|----------------------------------|
| SensorTask | Read DHT22, LDR/ADC, and sensor snapshot | 2 s | 2 | Separate display and alarm queues | <code>`vTaskDelayUntil()`</code> |
| DisplayTask | Own and update the OLED | Sensor, page, or state event | 1 | Queue set over three queues | Waiting on queue set |
| InputTask | Decode encoder and select page | 10 ms | 3 | Display-mode queue; reads ACTIVE event bit | <code>`vTaskDelayUntil()`</code> |
| MotionTask | Sample PIR and update inactivity state | 100 ms | 3 | Event group and system-state queue | <code>`vTaskDelayUntil()`</code> |
| AlarmTask | Evaluate temperature and control buzzer | Sensor queue update | 2 | Alarm sensor queue and ALARM event bit | Waiting for sensor data |

Priority rationale. MotionTask receives prompt PIR sampling and timeout response; InputTask keeps page navigation responsive. SensorTask and AlarmTask perform periodic/queued data processing. DisplayTask can tolerate update latency. These priorities give motion/input a higher chance to run when ready without keeping them continuously active; their periodic/blocking behavior still permits lower-priority work.

Sensor snapshots are sent to independent one-item queues so DisplayTask and AlarmTask each receive the latest sample. A queue set allows DisplayTask to block on data, mode changes, and state transitions. The event group uses `EVENT\_ACTIVE` (bit 0), `EVENT\_MOTION` (bit 1), and `EVENT\_ALARM` (bit 2). MotionTask produces ACTIVE/MOTION changes; AlarmTask produces ALARM; InputTask and DisplayTask consume the relevant bits. A mutex protects task-context serial messages.

FreeRTOS Task Communication

```

```mermaid
flowchart LR
 SensorTask -->|SensorData overwrite| DisplaySensorQueue --> DisplayTask
 SensorTask -->|SensorData overwrite| AlarmSensorQueue --> AlarmTask
 InputTask -->|DisplayMode| DisplayModeQueue --> DisplayTask
 MotionTask -->|SystemState transition| SystemStateQueue --> DisplayTask
 MotionTask -->|set/clear ACTIVE and MOTION| EventGroup[(Event group)]
 AlarmTask -->|set/clear ALARM| EventGroup
 EventGroup -->|read ACTIVE| InputTask
 EventGroup -->|read state bits| DisplayTask
```

```

Figure 3. Sensor data and commands travel through queues; shared state is signaled through event-group bits. DisplayTask alone owns the OLED.

State Machine

```

```mermaid
stateDiagram-v2
 [*] --> ACTIVE
 ACTIVE --> INACTIVE: no PIR motion for 15 seconds
 INACTIVE --> ACTIVE: PIR motion detected
 note right of ACTIVE
 OLED on
 encoder active
 sensors and alarm active
 end note
 note right of INACTIVE
 OLED off
 encoder ignored
 PIR monitoring continues
 end note
```

```

...

Figure 4. MotionTask owns the inactivity transition. DisplayTask applies OLED power changes; it is the sole OLED owner.

4. Implementation

- Sensor acquisition: DHT22 communication uses bounded pulse waits and checksum validation. LDR acquisition uses ADC1 channel 0. The raw count is mapped to a percentage over the ADC full scale; this is deliberately not labeled lux.
- Periodic scheduling: SensorTask samples every 2000 ms using ``vTaskDelayUntil()``. This anchors execution to a periodic wake schedule rather than adding the duration of every read to the next period.
- Display and input: DisplayTask alone initializes, writes, and powers the OLED. InputTask decodes quadrature movement and sends a ``DisplayMode`` enum. Clockwise order is Temperature, Humidity, Light, Motion, then Temperature.
- Alarm: ``evaluateTemperature()`` is a pure decision function. Below 18 C returns `LOW_TEMPERATURE`, above 30 C returns `HIGH_TEMPERATURE`, and both exact boundaries plus values between them return `NORMAL`. AlarmTask separately controls the buzzer and event bit.
- System state: ``evaluateSystemState()`` isolates `ACTIVE/INACTIVE` decisions. Motion returns the system to `ACTIVE`; no motion for 15 seconds transitions it to `INACTIVE`.
- Concurrency: Queues transfer snapshots and commands instead of using unsynchronized global sensor data. A mutex serializes complete task log messages. The event group communicates current shared status bits.
- Main entry point: ``main.cpp`` follows hardware initialization, RTOS object creation, task creation, and scheduler start. Task bodies reside in separate modules.

5. Verification and Testing

Requirements Traceability

The matrix follows the requirement identifiers shown in the laboratory handout. FT references are planned Wokwi checks; until their actual/result fields are completed in ``test/verification.md``, they indicate intended verification coverage rather than a functional PASS claim.

| Requirement | Implementation | Verification evidence |
|-------------|--|--|
| FR-01 | SensorTask and DHT22 acquisition | FT-01 temperature and FT-02 humidity (Wokwi observation pending). |
| FR-05 | DisplayTask and OLED sensor pages | FT-01 through FT-03 (Wokwi observation pending). |
| FR-06 | InputTask, DisplayMode enum, and navigation logic | Four native navigation unit tests passed; FT-04/FT-05 Wokwi observation pending. |
| FR-07 | AlarmTask and <code>`evaluateTemperature()`</code> | Five native temperature-alarm tests passed; FT-06/FT-07 Wokwi observation pending. |

| FR-09 | MotionTask and system-state logic | Four native state tests passed; FT-08 through FT-10 Wokwi observation pending. |

Unit Tests

The native Unity suite contains 13 deterministic tests and the user-observed test output reports 13 test cases, 13 succeeded:

- Temperature alarm: below lower threshold, exact lower threshold, normal value, exact upper threshold, above upper threshold (5 tests).
- Display navigation: forward, reverse, Motion-to-Temperature wrap, Temperature-to-Motion reverse wrap (4 tests).
- System state: ACTIVE without timeout, ACTIVE with timeout, INACTIVE without motion, and motion reactivation (4 tests).

Run using ``pio test -e native``. These tests cover decision logic; they do not test physical or simulated HAL timing.

Build and Static Analysis

The Blue Pill firmware builds with ``pio run -e bluepill_f103c8``. PlatformIO Check was run with ``pio check -e bluepill_f103c8``. The full scan reported low-severity style/integration notices in CubeMX HAL callbacks, interrupt handlers, and the FreeRTOS heap implementation. Medium- and high-severity scans were clear. The notices and rationale for retaining framework entry points are recorded in ``test/verification.md``.

Wokwi Functional Verification

The functional test plan is recorded in ``test/verification.md`` as FT-01 through FT-10. Actual/result entries remain pending until each behavior is observed in Wokwi. Expected checks include updated temperature/humidity/light pages, forward/reverse encoder navigation, alarm activation above 30 C and clearing in the normal range, PIR activity, inactivity after 15 seconds, and motion-triggered reactivation.

Required visual evidence: Insert a screenshot captured from the running Wokwi simulation here after the system is verified. Caption: *Figure 5. Finished multisensor monitor showing the OLED and live simulation.*

Deliberate Fault Experiments

Three compile-time switches in ``include/fault_experiments.h`` default to zero. They respectively remove SensorTask's delay, raise InputTask to the highest priority, and bypass the serial mutex. Only one should be enabled at a time; it must be restored to zero and rebuilt afterward. Fault observations have not yet been recorded and must not be reported as completed. In particular, removing the sensor delay can stall this custom scheduler, and removing the mutex may not visibly interleave output because UART writes are synchronous and do not yield mid-message.

6. Static Code Analysis

`pio check -e bluepill\_f103c8` reports only low-severity findings:

| Finding | Location | Interpretation and action |
|-------------------------|---|---|
| --- | --- | --- |
| `constParameterPointer` | `src/cubemx/stm32f1xx_hal_msp.c` | callback parameters HAL callback signatures are framework-defined; retained to match those interfaces. |
| `unusedFunction` | CubeMX MSP, HAL timebase, and interrupt-handler files | Cppcheck cannot see framework calls or startup-vector references; retained as required entry points. |
| `unusedFunction` | `src/heap_4.c` | allocation and heap APIs Kernel allocation is called indirectly; optional heap-stat functions are not used by this application. Retained from the FreeRTOS allocator. |
| `variableScope` | `src/heap_4.c` | allocator local Style-only suggestion in allocator code; no behavioral defect identified. |

No medium- or high-severity defects were reported. Details and source locations are in `test/verification.md`.

7. Engineering Discussion

Trade-offs

- A one-element overwrite queue favors the newest sensor reading and bounded memory over retaining every sample. This suits a live display, but it is not an event-history log.
- Separate queues duplicate a small `SensorData` snapshot so consumers do not compete for one message.
- Higher priority for input/motion reduces response latency when those tasks are ready. Excessive priority or a task that fails to block can delay lower-priority work.
- OLED single ownership avoids concurrent I2C updates and removes the need for an OLED mutex.
- LDR normalization makes readings understandable without claiming unsupported lux calibration.
- The custom FreeRTOS context-switch port is simulation-specific and requires careful handling of blocking/yield paths.

Limitations and Debugging

- Wokwi functional outcomes remain to be entered in the verification record. A successful build/unit test does not prove sensor wiring or simulator behavior.
- PIR timeout depends on the PA3 input returning LOW; a continuously HIGH input continually restarts the inactivity timer.
- Native tests do not simulate I2C transactions, DHT pulse timing, ADC behavior, PWM, or task scheduling on the MCU.
- Static analysis cannot follow all HAL callback and interrupt-vector references, producing expected low-severity unused-function notices.
- Fault experiments require observation and restoration; no outcome is claimed until recorded.

Alternatives Considered

Queues were selected instead of shared sensor globals to define ownership and synchronization. Event bits represent current state while queues carry data snapshots and transitions. A queue set lets DisplayTask block on multiple event sources. Hardware-independent pure functions were extracted to make boundary and state-transition tests runnable on a host.

8. Conclusion

The project demonstrates a modular FreeRTOS sensor monitor with queue-based data distribution, event-group state signaling, priority-aware task design, serialized logging, an OLED display, encoder navigation, alarm logic, and a PIR inactivity state machine. The 13 native logic tests passed and the embedded firmware builds. Completion of the functional verification and fault-experiment sections still depends on recording actual Wokwi observations and adding the required screenshots.

References and Acknowledgments

- FreeRTOS documentation: <https://www.freertos.org/Documentation/00-Overview>
- PlatformIO documentation: <https://docs.platformio.org/>
- Wokwi documentation: <https://docs.wokwi.com/>
- STM32F1 HAL and CMSIS support are provided through the STM32Cube framework.